Guide for judges: “Is the Map Right?”

This page tells you where every part of the paper comes from and how to check it. All paths are relative to `planA_results/` in the repository. Everything below is also packed in `papers/supplement_paper1.zip`, except the two largest folders noted at the end.

1. Read

What	File
Paper (PDF)	`papers/latex/paper1_is_the_map_right.pdf`
Paper (LaTeX source)	`papers/latex/paper1_is_the_map_right.tex`
Paper (Markdown, same text)	`papers/paper1_is_the_map_right.showdown.md`
Figures (vector / PNG)	`papers/figures/*.pdf`, `papers/figures/png/*.png`
Numbers behind each figure	`papers/figures/figure_data.json`

2. Check the numbers in one command

```
cd planA_results
python3 papers/verify_paper1_numbers.py
```

Standard library only, a few seconds. It re-reads the result files and compares 66 quantitative claims in the paper with the values in the files, printing PASS or FAIL with the source file of each. The saved output is `papers/verify_paper1_numbers.out.txt`: **63 PASS, 3 FAIL, 9 INFO** (INFO lines list values that appear only in figures).

The three FAIL lines are real discrepancies in the paper text, kept visible rather than hidden:

Paper says	Files say	Size of the effect
“Every missing nested definition in fastapi and rich is itself async” (§5.1)	rich has one non-async missing nested definition, a class two levels deep in `tests/test_inspect.py`, absent in each of its 47 graphs	One test-only definition; fastapi holds
“8 to 21% of `calls` edges per graph point from a symbol to its own member” (§5.4)	8% to 21% in 126 graphs; the single httpx graph is 29.8%	One graph of 127
“pairwise rank correlation at most 0.16” (§5.6)	maximum over all variants is 0.17 for EmbeddingGemma 300m, 0.14 for EmbeddingGemma 2, 0.21 for the MiniLM control	No candidate approaches identification either way (best mean cosine 0.073)

3. Where each section's results live

Paper section	Result files	Produced by
§5.1 Omission (Fig. 1)	`vm-a2/results/a1_v2/a1_summary.json`, `a1_tasks.csv`, `a1_gold_symbols.jsonl`, `a1_graphs.csv`	`scripts/planA/a1_census.py`
§5.1 Id reconciliation	`vm-a2/results/a7/a7_summary.json`, `a7_class_dropped.csv`, `a7_provided_nodes.csv`, `a7_defs_absent.csv`	`scripts/planA/a7_id_audit.py`
§5.2 Resolver (Fig. 2)	`vm-a2/results/a1_v2/a1_summary.json` (`misresolution`)	`scripts/planA/a1_census.py`, resolver copy in `scripts/planA/common.py`
§5.3 Recall and precision (Table 1, Fig. 3)	`vm-a2/results/a2_v3/a2_summary.json`, `a2_tasks.csv`, `a2_run_status.csv`, `a2_dyn_edges.jsonl`, `a2_static_unconfirmed.jsonl`	`scripts/planA/a2/run_task.py`, `trace_plugin.py`, `analyze.py`
§5.4 Edge semantics (Figs. 4 and 5)	`vm-a2/results/a8/a8_summary.json`, `a8_edges.csv`, `a8_sample.csv` (80 edges with caller snippets)	`scripts/planA/a8_edge_audit.py`; structure classes in `papers/make_figures.py`
§5.5 Reachability (Fig. 6)	`vm-a2/results/a2_v3/a2_summary.json` (`gold_min_hops_from_entry_hist`), `a2_gold.jsonl`	`scripts/planA/a2/analyze.py` (`entry_nodes`)
§5.6 Encoder	`vm-a2/results/a3/`, `vm-a2/results/a3_gemma2/`	`scripts/planA/a3_encoder.py`
§5.6 Similarity (Fig. 7)	`vm-main/results/a4/a4_summary.json`, `a4_relations.csv`, `a4_knn.csv`	`scripts/planA/a4_probe.py`
§5.6 Retrieval (Fig. 8)	`vm-a2/results/a5_v3/a5_summary.json`, `a5_tasks.csv`	`scripts/planA/a5_retrieval.py`
§6 Released graph and profile	`vm-a2/results/a6_v3/graphs_v2/*.json` (one per commit), `a6_v3/task_profile.csv`	`scripts/planA/a6_release.py`
Per-task tests and traces	`vm-a2/results/a2_runs/<task>/` (status, pytest logs, junit XML, traces, dependency logs)	`scripts/planA/a2/run_task.py`

4. Reproduce from scratch

The competition archive (tasks, graphs, embeddings, snapshots) is not redistributed here; download it from the competition. Then, on a Linux machine with Python 3.13 and network access:

```
cd planA_results/scripts
python3 planA/run_all.py --data <archive> --out <results> --stages a1,a4,a5 --a1-shards 8 --jobs 8
python3 planA/a2/run_task.py --data <archive> --out <results>/a2_runs --backend venv --jobs 16 --keep
bash ops/rerun_fixes.sh <archive> <results> <cache> 8
bash ops/run_gaps.sh <archive> <results> <cache>
```

Measured run times (from the step logs): census 2.4 minutes on 8 shards; the dynamic oracle 9,756 task-seconds in total (median 57 s per task, so roughly 10 minutes of wall time at 16 parallel jobs, plus up to 25 minutes for a task whose tests hang); every analysis step under 5 minutes. The encoder step needs a Hugging Face token with access to EmbeddingGemma. Host specs and package versions are in `provenance/`; script checksums for each pass are in `vm-a2/results/scripts_sha256_v3.txt`.

Tests of the pipeline itself (synthetic fixture, no competition data needed):

```
cd planA_results/scripts && python3 -m pytest tests/test_pipeline.py -q
```

15 tests pass.

5. History and caveats worth knowing

- The first run (Oct 10) used a wrong id scheme for nested definitions; the fix and every changed number are documented in `FIXES_2026-10-11.md`, and the final numbers in `ANALYSIS_2026-10-11_VM_RERUN.md`. The paper uses only the final outputs (`a1_v2`, `a2_v3`, `a5_v3`, `a6_v3`, `a7`, `a8`).
- One task (rich_4006) hung on four test cases; it was recovered by deselecting them and recording them as failed (`vm-a2/results/a2_runs/rich_4006/status.json`, `hung_tests`).
- 15 of 129 tasks did not run cleanly in our environment (9 collection errors, 2 import errors, 4 requests tasks with no collectable tests). These reflect our dependency setup, not the tasks.

6. Not in the zip

`vm-a2/results/a6_v3/graphs_v2/` (662 MB, the released typed graphs; one example is included) and `vm-a2/results/a1_v2/a1_async_audit.csv.gz` (per-definition census rows). Both are in the repository.